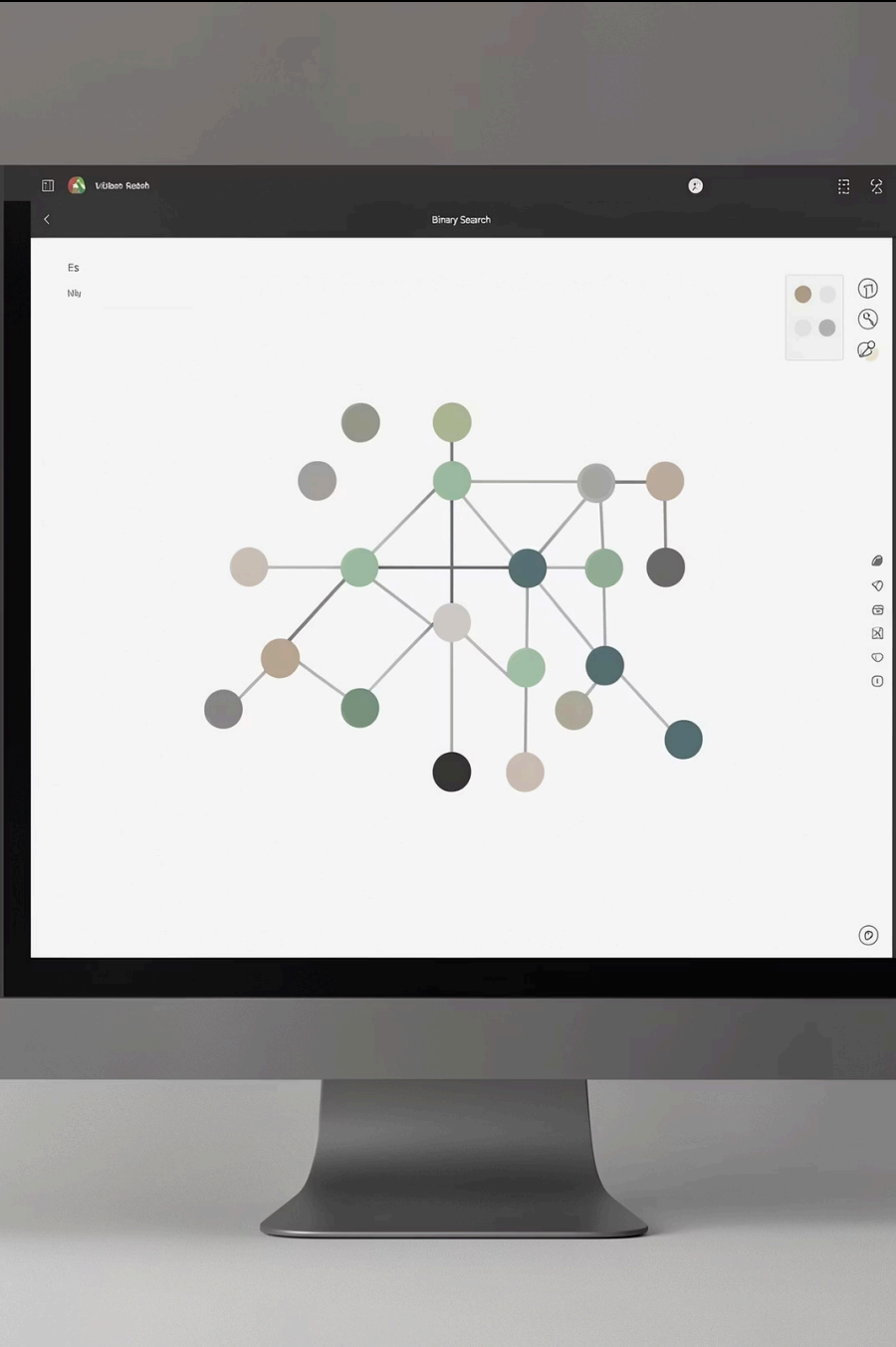




Métodos de Búsqueda: Búsqueda Binaria

Conoce, analiza y compara el funcionamiento de los diferentes métodos de búsqueda, específicamente el algoritmo binario, para la solución de problemas que así lo requieran.

Por Taddheo Torres y Francisco Javier

¿Qué es la Búsqueda Binaria?

La **búsqueda binaria** es un algoritmo eficiente para encontrar un elemento específico en una **lista ordenada**. Funciona dividiendo repetidamente el espacio de búsqueda a la mitad, eliminando la mitad donde el elemento objetivo no puede estar.

Es como buscar en un diccionario: abres por la mitad y decides si ir adelante o atrás. Así el algoritmo divide el problema a la mitad cada vez, haciéndolo mucho más rápido.

Este método es **mucho más rápido** que la búsqueda lineal, especialmente para conjuntos de datos grandes. Su eficiencia se debe a que en cada paso reduce el espacio de búsqueda por un factor de 2.



Requisito Principal

La lista debe estar ordenada de manera ascendente o descendente

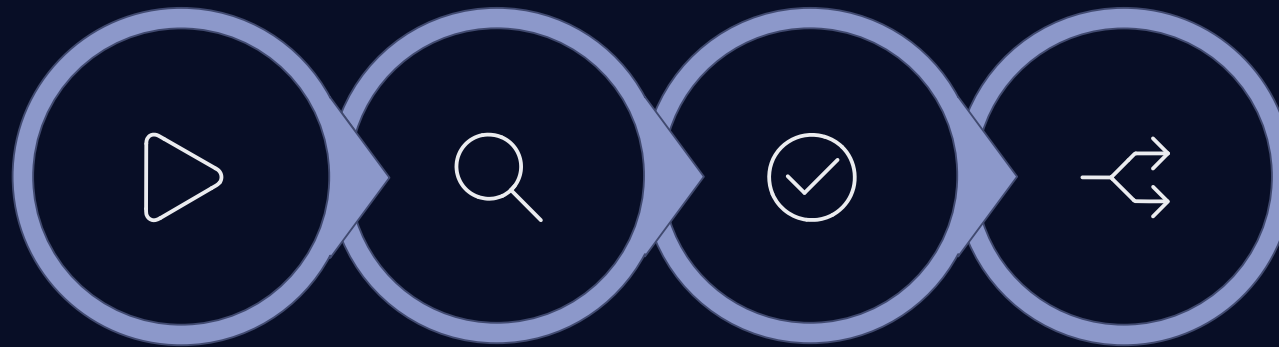
Enfoque Dividir y Conquistar

Divide el problema en subproblemas más pequeños hasta encontrar la solución

¿Cómo Funciona?

Compara el número buscado con el del medio. Si es mayor, descarta la mitad izquierda. Si es menor, descarta la derecha. Repite hasta encontrarlo.

Cada paso elimina la mitad. En 1,000 elementos: máximo 10 comparaciones. En 1 millón: solo 20. Por eso es tan eficiente.



Inicio

Comparar

Coincide?

Izquierda/Derecha

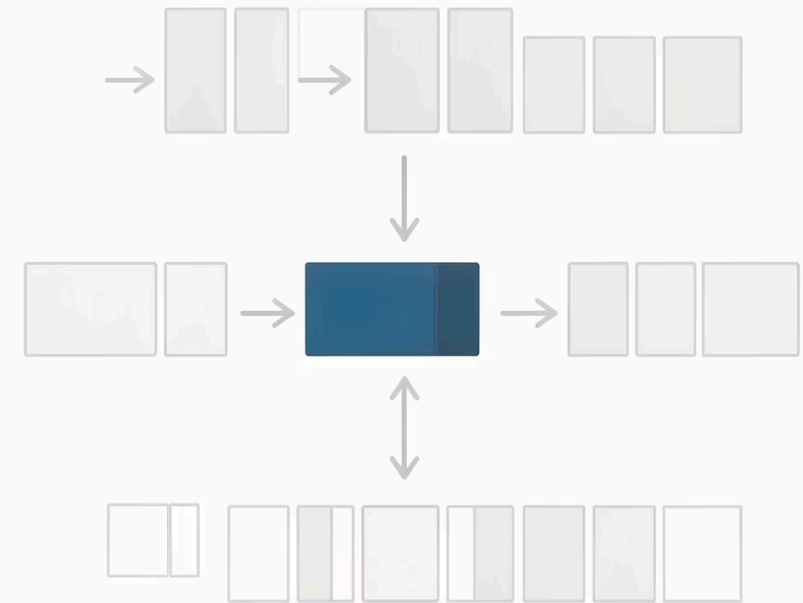
Ejemplo Práctico

Busquemos el número **42** en esta lista ordenada:



En este caso, el algoritmo encontró el elemento en la **primera comparación**. En el peor caso, necesitaría $\log_2(n)$ comparaciones, donde n es el tamaño de la lista.

Si buscamos 78: comparamos con 42 (derecha), luego 67 (derecha), luego 78 (¡encontrado!). Solo 3 pasos. Con búsqueda lineal serían 9.



Implementación en Código

Usas dos punteros (izquierda y derecha). Calculas el medio, comparas, y mueves un puntero según el resultado. Repites hasta encontrar el elemento.

Java

```
public static int busquedaBinaria(int[] array, int objetivo) {
    int izquierda = 0;
    int derecha = array.length - 1;

    while (izquierda <= derecha) {
        int medio = izquierda + (derecha - izquierda) / 2;

        if (array[medio] == objetivo) {
            return medio; // Elemento encontrado
        }

        if (array[medio] < objetivo) {
            izquierda = medio + 1; // Buscar en mitad derecha
        } else {
            derecha = medio - 1; // Buscar en mitad izquierda
        }
    }

    return -1; // Elemento no encontrado
}
```

C++

```
int busquedaBinaria(int array[], int objetivo, int n) {
    int izquierda = 0;
    int derecha = n - 1;

    while (izquierda <= derecha) {
        int medio = izquierda + (derecha - izquierda) / 2;

        if (array[medio] == objetivo)
            return medio;

        if (array[medio] < objetivo)
            izquierda = medio + 1;
        else
            derecha = medio - 1;
    }

    return -1;
}
```

Pseudocódigo y Diagrama

Pseudocódigo

```
FUNCIÓN BusquedaBinaria(array,
objetivo):
  izquierda ← 0
  derecha ← LONGITUD(array) - 1

  MIENTRAS izquierda ≤ derecha:
    medio ← izquierda + (derecha -
    izquierda) / 2

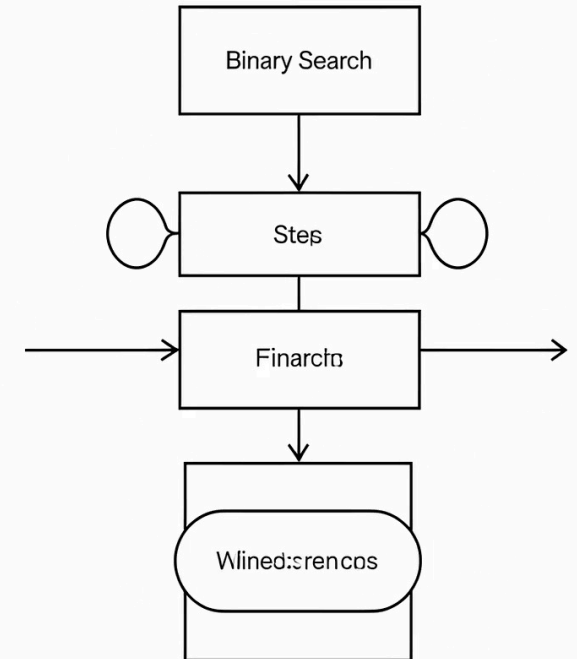
    SI array[medio] == objetivo:
      RETORNAR medio

    SI array[medio] < objetivo:
      izquierda ← medio + 1
    SINO:
      derecha ← medio - 1

  RETORNAR -1
FIN FUNCIÓN
```

Diagrama de Flujo

El diagrama muestra el flujo del algoritmo desde la inicialización hasta la verificación del punto medio, la comparación con el objetivo, y la actualización de los límites de búsqueda.



Ventajas y Desventajas

Es muy eficiente pero necesita datos ordenados. Si tu lista cambia constantemente, el costo de ordenarla puede no valer la pena.

Ventaja: Eficiencia Superior

Tiempo logarítmico $O(\log n)$: Consume significativamente menos tiempo que la búsqueda lineal $O(n)$. Por ejemplo, en una lista de 1 millón de elementos, necesita máximo 20 comparaciones versus 1 millón.

Ventaja: Menor Consumo de Recursos

Optimiza recursos computacionales: Al eliminar la mitad del espacio de búsqueda en cada paso, reduce drásticamente el número de operaciones necesarias.

Desventaja: Requiere Ordenación

Pre-procesamiento obligatorio: La lista debe estar ordenada previamente. Ordenar una lista desordenada consume $O(n \log n)$ tiempo, lo cual puede ser peor que usar búsqueda lineal.

Desventaja: Ineficiente en Estructuras Dinámicas

No es óptimo para listas que cambian frecuentemente: Cada inserción o eliminación puede requerir re-ordenar, lo que hace que otros algoritmos sean más apropiados.

Comparación con Otros Algoritmos

Algoritmo	Tiempo	Requisito
Búsqueda Lineal	$O(n)$	Ninguno
Búsqueda Binaria	$O(\log n)$	Ordenado
Búsqueda Hash	$O(1)$	Tabla hash

Video del Funcionamiento del Algoritmo

Visualiza cómo el algoritmo de búsqueda binaria divide el espacio de búsqueda en cada iteración, eliminando eficientemente la mitad donde el elemento objetivo no puede estar.

📄 **Nota:** Espacio reservado para integrar el video del funcionamiento del algoritmo (máximo 6 minutos). Puedes añadir tu contenido multimedia aquí.

